COMPTIA SECURITY+ SY0-701 | DOMÍNIO 2 · AMEAÇAS, VULNERABILIDADES E MITIGAÇÕES (22%)

Security+ 2.3 · Memory Injections

Mestre CyberSec · **Aula 2.3 — Memory Injections** (2:39) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Resumo Executivo da Aula: DLL injection: força processo legítimo a carregar biblioteca maliciosa; process hollowing: substitui a memória de um processo legítimo rodando. Antivírus por assinatura não pega (o processo é 'legítimo'), só EDR behavioural.

💡 Highlights que a prova CompTIA cobra

- DLL injection: processo carrega DLL do atacante
- Process hollowing: shellcode substitui código de processo suspendido

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'malicious code runs inside a legitimate process's memory' = process hollowing / DLL injection

🚨 Além do Vídeo: Conceitos Oficiais, Siglas e Armadilhas (Seção 2.3 — Memory Injections)

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Injeção de DLL

Ataque que força um processo legítimo em execução a carregar uma Dynamic-Link Library maliciosa, fazendo código do atacante rodar dentro de uma memória que aparenta ser confiável.

⚠ ***Armadilha CompTIA:*** *Gatilho: 'processo legítimo CARREGA/é induzido a carregar uma biblioteca maliciosa' = DLL injection. O processo original continua rodando — não confunda com substituição de processo (hollowing).*

• Process Hollowing (Esvaziamento de Processo)

Ataque em que um processo legítimo é iniciado em estado suspenso, seu código original é removido da memória e um shellcode malicioso é escrito em seu lugar antes da execução ser retomada — o código malicioso roda como se fosse um processo confiável.

⚠ ***Armadilha CompTIA:*** *Gatilho: 'código de processo legítimo é SUBSTITUÍDO por código malicioso' ou 'processo suspenso esvaziado' = process hollowing. Distinção-chave: hollowing substitui o processo; DLL injection apenas adiciona/carrega uma biblioteca em um processo vivo.*

• Biblioteca de Vínculo Dinâmico (DLL)

Biblioteca compartilhada do Windows contendo código e dados que vários programas carregam e chamam em tempo de execução; é o veículo que atacantes abusam para injetar código malicioso em processos legítimos.

⚠ ***Armadilha CompTIA:*** *A prova pode descrever DLL como 'shared library'. A DLL em si é legítima — a maldade está no atacante forçar sua carga em um processo confiável, evadindo verificação por assinatura baseada em arquivo.*

• Shellcode

Pequeno payload de código de máquina nativo injetado diretamente na memória de um processo e executado, tipicamente usado para substituir código legítimo ou executar instruções controladas pelo atacante.

⚠ ***Armadilha CompTIA:*** *No process hollowing, o shellcode é o que substitui o código do processo suspenso. Se a questão menciona 'código nativo escrito nos espaços do processo', pense em shellcode/hollowing.*

• Detecção e Resposta em Endpoint (EDR)

Solução de monitoramento que analisa continuamente o comportamento do endpoint, incluindo processos, atividade de memória e chamadas de API, para detectar e responder a ameaças como DLL injection e process hollowing que evadem ferramentas por assinatura.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'detecção COMPORTAMENTAL' ou 'detecta código malicioso rodando dentro de processo legítimo' = EDR. Na prova, EDR/behavioral é a detecção correta para memory injection; antivírus por assinatura é o distrator errado.*

• Detecção por Assinatura

Método de detecção que compara arquivos e código com hashes ou padrões conhecidos de malware; falha contra memory injections porque o código malicioso roda dentro da memória de um processo legítimo (e assinado).

⚠️ ***Armadilha CompTIA:*** *A prova inverte o óbvio: o processo é 'legítimo', então assinatura NÃO detecta. Se a pergunta perguntar por que o antivírus falhou contra memory injection, a resposta é que a detecção por assinatura não analisa o comportamento em memória.*

• Malware Sem Arquivo (Fileless)

Código malicioso que reside apenas na memória (RAM) e usa aplicativos legítimos ou ferramentas nativas para operar, deixando poucos ou nenhum artefato em disco para o antivírus tradicional analisar.

⚠️ ***Armadilha CompTIA:*** *Memory injections são uma forma de ataque fileless: o código vive na memória de processos legítimos, não no disco. Gatilho 'não deixa rastros no disco / roda só em memória' = fileless, exige detecção comportamental (EDR).*

• Escala de Privilégio

Objetivo do atacante de obter direitos mais altos ou contexto confiável de execução rodando código malicioso dentro de processos legítimos, burlando controles de segurança que monitoram aplicativos não confiáveis.

⚠️ ***Armadilha CompTIA:*** *DLL injection e process hollowing são TÉCNICAS usadas para escalar privilégio/evadir defesa: rodar no contexto de processo confiável concede acessos extras. Se a questão pede o OBJETIVO → privilege escalation; se pede a TÉCNICA → hollowing ou injection.*

✅ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

Um analista de segurança observa que WINWORD.EXE, que é assinado e esperado, fez com que um módulo não assinado chamado update.dll fosse carregado dentro do EXPLORER.EXE. A telemetria do EDR também mostra chamadas WriteProcessMemory e CreateRemoteThread de WINWORD.EXE para EXPLORER.EXE. O antivírus baseado em assinatura não alerta. Qual técnica de injeção de memória provavelmente está sendo usada?

A) Process hollowing

B) Injeção de DLL

C) Sequestro de ordem de busca de DLL

D) Carregamento reflexivo de DLL

QUESTÃO 2 · CENÁRIO DE EXAME

Durante a resposta a incidentes, um analista descobre que um processo de sistema assinado foi criado em estado suspenso. Sua imagem de memória original foi desmapeada, código malicioso foi escrito no espaço de endereço vazio e o processo foi retomado. O executável no disco ainda parece legítimo. Qual técnica isso descreve?

A) Injeção de DLL

B) Sequestro de execução de thread

C) Process hollowing

D) Injeção remota de shellcode

QUESTÃO 3 · CENÁRIO DE EXAME

O antivírus baseado em assinatura de uma organização não reporta infecção em um endpoint. No entanto, o EDR mostra código malicioso em execução dentro da memória de um aplicativo confiável e assinado digitalmente, e nenhum executável malicioso é encontrado no disco. Qual fator melhor explica por que o antivírus não detectou a atividade?

A) O mecanismo antivírus foi desabilitado por política de grupo.

B) O malware criptografou o disco inteiro antes de executar.

C) O aplicativo confiável bloqueou todas as varreduras do sistema de arquivos.

D) O código malicioso é executado em um processo confiável e pode não existir como um arquivo com assinatura reconhecível.

QUESTÃO 4 · CENÁRIO DE EXAME

Uma equipe de segurança quer melhorar a detecção de injeção de DLL e process hollowing em endpoints Windows. Qual capacidade é mais apropriada para identificar esses comportamentos?

A) Verificação do sistema de arquivos por antivírus baseado em assinatura.

B) Monitoramento por EDR de memória de processos, chamadas de API e relações entre processos pai e filho.

C) Regras de firewall de host que bloqueiam a porta de saída 445.

D) Criptografia de disco completa com chaves respaldadas por TPM.

QUESTÃO 5 · CENÁRIO DE EXAME

Um processo de usuário com baixo privilégio injeta uma DLL maliciosa em um serviço Windows que está sendo executado sob a conta SYSTEM. O código injetado começa a executar dentro desse serviço. Qual é o impacto de segurança mais significativo?

A) O código malicioso pode ser executado com privilégios de nível SYSTEM.

B) A assinatura digital do serviço é automaticamente revogada.

C) A DLL é copiada para o perfil SYSTEM para backup.

D) O hash da senha do usuário é redefinido imediatamente.

QUESTÃO 6 · CENÁRIO DE EXAME

Os logs do EDR mostram um processo chamando OpenProcess em um processo remoto, depois VirtualAllocEx, WriteProcessMemory e CreateRemoteThread. Os bytes gravados são código diretamente executável, não um caminho para uma biblioteca, e nenhum novo executável ou DLL é gravado no disco. Qual atividade melhor corresponde a essa sequência?

A) Process hollowing

B) Sequestro de ordem de busca de DLL

C) Coleta de credenciais do LSASS

D) Injeção remota de shellcode

QUESTÃO 7 · CENÁRIO DE EXAME

Um alerta de EDR afirma que a imagem de memória de um processo em execução não corresponde ao executável assinado armazenado no disco. O processo está usando um nome legítimo, como svchost.exe. Qual técnica isso é o indicador mais forte?

A) Injeção de DLL

B) Sequestro de ordem de busca de DLL

C) Process hollowing

D) Keylogging por dispositivo de hardware

QUESTÃO 8 · CENÁRIO DE EXAME

Uma empresa aplica uma política de allowlisting de aplicativos que permite apenas executáveis aprovados e assinados. Um malware faz com que um executável assinado carregue uma DLL não assinada de uma pasta gravável pelo usuário, e a DLL é executada. Qual controle de segurança foi contornado?

A) Allowlisting de aplicativos

B) Segmentação de rede

C) Autenticação multifator

D) Criptografia de disco completa

QUESTÃO 9 · CENÁRIO DE EXAME

Um analista confirma que uma estação de trabalho está executando um processo hollowed contendo código malicioso. A estação ainda está conectada à rede corporativa. Qual ação melhor apoia contenção e preservação de evidências?

A) Reiniciar imediatamente a estação para limpar a memória maliciosa.

B) Isolar o endpoint da rede e capturar a memória volátil antes de encerrar o processo.

C) Executar uma varredura antivírus completa por assinatura e continuar usando a estação.

D) Excluir todos os arquivos da pasta temporária do usuário e fechar o processo.

QUESTÃO 10 · CENÁRIO DE EXAME

Um aplicativo Windows é vulnerável a injeção de memória. Os administradores querem dificultar a execução de shellcode injetado em regiões de memória somente de dados. Qual mitigação de exploração do Windows é mais relevante?

A) Criptografia de unidade BitLocker

B) Delegação restrita do Kerberos

C) Extensões de Segurança do DNS (DNSSEC)

D) Prevenção de Execução de Dados (DEP)

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa B — Injeção de DLL

A injeção de DLL ocorre quando um processo legítimo é forçado a carregar uma biblioteca maliciosa. O uso de WriteProcessMemory e CreateRemoteThread para direcionar outro processo, seguido do carregamento de update.dll, corresponde a injeção de DLL clássica. Process hollowing substituiria a imagem do processo, não apenas adicionaria um módulo. DLL search order hijacking explora a ordem de busca de DLLs, não necessariamente escrita remota e criação de thread. Reflective DLL loading normalmente mapeia uma DLL diretamente da memória, sem gravar um caminho de arquivo para LoadLibrary.

Questão 2: Alternativa C — Process hollowing

Process hollowing cria um processo legítimo em estado suspenso, remove ou desmapeia sua imagem original, escreve código malicioso em seu espaço de memória e retoma a execução. O processo mantém aparência legítima, enquanto o código em memória é malicioso. Injeção de DLL adiciona uma biblioteca a um processo existente; thread execution hijacking altera a execução de uma thread; injeção remota de shellcode insere shellcode sem necessariamente desmapear a imagem original.

Questão 3: Alternativa D — O código malicioso é executado em um processo confiável e pode não existir como um arquivo com assinatura reconhecível.

Antivírus baseado em assinatura depende de arquivos ou padrões conhecidos em disco. Em ataques de injeção de memória, como DLL injection e process hollowing, o código malicioso pode residir apenas na memória de um processo confiável e assinado. Assim, não há necessariamente um arquivo malicioso com assinatura detectável. As demais alternativas não explicam diretamente essa limitação.

Questão 4: Alternativa B — Monitoramento por EDR de memória de processos, chamadas de API e relações entre processos pai e filho.

EDR é eficaz contra injeção de memória porque monitora comportamento, chamadas de API suspeitas (por exemplo, WriteProcessMemory, CreateRemoteThread, NtUnmapViewOfSection), integridade da memória e relações entre processos. Antivírus por assinatura pode não detectar código apenas em memória. Firewall de host e criptografia de disco não detectam injeção em memória.

Questão 5: Alternativa A — O código malicioso pode ser executado com privilégios de nível SYSTEM.

Ao injetar código em um processo que roda como SYSTEM, o código geralmente herda o token e os privilégios desse processo. Isso permite escalação de privilégios, pois o atacante passa de baixo privilégio para execução como SYSTEM. A assinatura digital não é revogada, a DLL não é copiada para backup pelo sistema e a senha do usuário não é redefinida por esse mecanismo.

Questão 6: Alternativa D — Injeção remota de shellcode

A sequência OpenProcess, VirtualAllocEx, WriteProcessMemory e CreateRemoteThread é típica de injeção remota de shellcode: aloca memória no processo alvo, escreve o shellcode e cria uma thread para executá-lo. Como os bytes gravados são código executável, e não um caminho para biblioteca, e nenhum novo executável ou DLL é gravado no disco, não corresponde ao padrão mais direto de process hollowing ou carregamento de DLL. A coleta de credenciais do LSASS envolveria acesso à memória do LSASS, mas não necessariamente essa sequência exata.

Questão 7: Alternativa C — Process hollowing

Process hollowing é caracterizado por um processo legítimo cuja memória foi substituída por código malicioso. Por isso, a imagem em memória não corresponde ao executável assinado no disco. Injeção de DLL adiciona um módulo, mas não necessariamente substitui a imagem principal. DLL search order hijacking explora o carregamento de uma DLL errada, e keylogging por hardware não se relaciona a essa discrepância de imagem em memória.

Questão 8: Alternativa A — Allowlisting de aplicativos

Allowlisting de aplicativos normalmente autoriza executáveis, mas pode não controlar todas as DLLs carregadas por esses executáveis. Se um binário assinado e permitido carrega uma DLL não assinada de local gravável, o código malicioso pode executar sem que um novo executável não autorizado seja iniciado diretamente. Segmentação de rede, MFA e criptografia de disco não são contornadas por esse mecanismo específico.

Questão 9: Alternativa B — Isolar o endpoint da rede e capturar a memória volátil antes de encerrar o processo.

A memória volátil contém evidências críticas, como código injetado, módulos carregados e artefatos de processo. Isolar primeiro contém a propagação, e capturar a memória antes de encerrar o processo preserva evidências que seriam

perdidas. Reiniciar ou encerrar imediatamente destrói a memória; uma varredura por assinatura pode não detectar código apenas em memória; excluir temporários não remove a injeção em execução.

Questão 10: Alternativa D — Prevenção de Execução de Dados (DEP)
DEP marca regiões de memória como não executáveis, ajudando a impedir que código injetado em áreas de dados seja executado. BitLocker protege dados em repouso; Kerberos lida com autenticação e delegação; DNSSEC protege a integridade do DNS. Nenhuma dessas opções impede diretamente a execução de shellcode em memória como DEP.